
SLEEP: Simulated Future Learning Environments for Automated Evolution of Heuristic Portfolios

Nguyen Viet Tuan Kiet^{*1} Nguyen Huu Duc^{*1} Tran Cong Dao^{*2}
Bui Trong Duc¹ Huynh Thi Thanh Binh¹

Abstract

We introduce continual heuristic portfolio learning, a new setting of automatic heuristic design (AHD) for nonstationary combinatorial optimization, where instances arrive over time and the learner must maintain a compact set of executable heuristics for future distributions. This setting naturally leverages LLM code generation: candidate heuristics can be synthesized as programs, executed inside a solver, and improved from objective feedback. We propose SLEEP, a sleep-wake framework that learns to construct synthetic training environments for an inner automatic heuristic set design (AHSD) procedure. During sleep, SLEEP prepares simulated and replayed instances for an LLM-based evolutionary portfolio designer; during wake, real-instance performance provides delayed feedback. Rather than forecasting the next distribution explicitly, SLEEP learns environments that induce robust and complementary heuristics. We further introduce EoH-C, a counterfactual extension of EoH-S that incorporates recent wake replay during portfolio selection. Experiments on classical combinatorial optimization benchmarks, including TSP and VRP variants, demonstrate improved future-instance performance under distribution shift.

1. Introduction

LLMs for code generation. Recent LLMs exhibit strong in-context learning, adapting to new tasks from instructions and examples without parameter updates (Brown et al., 2020; Kojima et al., 2022). Code-capable LLMs extend this ability to executable program synthesis for code completion, debugging, and competitive programming (Chen

et al., 2021; Li et al., 2022; Rozière et al., 2024). Since generated code can be executed and evaluated, code generation naturally supports closed-loop refinement from objective feedback such as errors, runtime, and task-level performance (Shinn et al., 2023; Romera-Paredes et al., 2024).

Automatic heuristic design (AHD). AHD applies this view to CO: each generated program implements a heuristic component embedded in a fixed solver and evaluated on problem instances. EoH (Liu et al., 2024) combines LLM generation with evolutionary search, ReEvo (Ye et al., 2024) adds LLM-generated reflections, and MCTS-AHD (Zheng et al., 2025) improves exploration through tree search. More recently, MOTIF (Kiet et al., 2026a) broadens AHD toward multi-strategy solver design, while EoH-S (Liu et al., 2026) studies AHSD, optimizing a complementary portfolio rather than a single heuristic. However, these methods are mainly offline and stationary: they optimize on a fixed training distribution and assume it remains useful for future instances.

Real optimization workloads are often sequential and non-stationary. Instances arrive over time, the distribution may drift, and useful design experience should be reused rather than restarted from scratch. This suggests a continual learning view (Parisi et al., 2019; Wang et al., 2024): instead of one-shot heuristic design, the learner should maintain a portfolio that can adapt before future instances are observed. The challenge is that the learner must decide what to train on during periods without access to future instances.

We propose SLEEP, motivated by the observation that continual heuristic design has two timescales. Wake phases provide sparse but reliable evidence about the current workload: they reveal which heuristics solve arriving instances well. Sleep phases turn this evidence into preparation for the future, when no real instances are available but the system can still simulate, evaluate, and improve. A portfolio is essential because distribution shift rarely preserves a single dominant heuristic; different heuristics may specialize to different regimes, and best-of-portfolio execution exploits this complementarity without committing to one design. SLEEP therefore treats sleep not as offline retraining on past data, but as an active process of constructing environments that make the next portfolio more robust to future shifts.

^{*}Co-first authors ¹Hanoi University of Science and Technology, Hanoi, Vietnam ²University of Sydney, Sydney, Australia. Correspondence to: Bui Trong Duc <ducbt@soict.hust.edu.vn>, Huynh Thi Thanh Binh <binhht@soict.hust.edu.vn>.

Contributions. We make three contributions. 1 First, we formulate continual heuristic portfolio learning, a non-stationary extension of AHSD. 2 Second, we introduce SLEEP, which learns simulated future learning environments for an inner LLM-based portfolio designer. 3 Third, we propose EoH-C, a counterfactual extension of EoH-S that uses recent wake replay during portfolio selection. We evaluate SLEEP on classical CO problems, including TSP and VRP variants, and compare against EoH-S variants to demonstrate the gains from learned sleep generation and counterfactual portfolio evolution.

2. Preliminary

2.1. Automatic Heuristic Set Design

Automated heuristic set design (AHSD) extends AHD from searching for one heuristic to searching for a small complementary set of heuristics. Let $\mathcal{X}$ be an instance space and $\mathcal{H}$ be a heuristic search space. For a fixed solver, the quality of heuristic $h \in \mathcal{H}$ on instance $x \in \mathcal{X}$ is denoted by $f(x, h)$. Without loss of generality, smaller values are better.

Given a training set $\mathcal{T} = \{x_i\}_{i=1}^N$ and a portfolio budget M , AHSD seeks a portfolio $P \subset \mathcal{H}$ with $|P| \leq M$ such that each instance can be handled by at least one heuristic in the set. Its empirical objective is the best-of-portfolio loss:

$$P^* \in \arg \min_{P \subset \mathcal{H}, |P| \leq M} \frac{1}{N} \sum_{i=1}^N \min_{h \in P} f(x_i, h). \quad (1)$$

This objective differs from standard AHD, which corresponds to the special case $M = 1$. It rewards complementarity: adding a heuristic is valuable when it improves the portfolio on instances where the current set is weak.

EoH-S (Liu et al., 2026) instantiates AHSD with an LLM-driven evolutionary search framework. Each heuristic is represented by a natural-language thought and executable code. Instead of using only average performance as fitness, EoH-S tracks instance-wise performance vectors and uses complementary population management and memetic search to evolve a set of heuristics with diverse strengths.

2.2. Continual Heuristic Portfolio Learning

We extend the AHSD setting above to a nonstationary online stream. At round t , wake instances are drawn from an unknown distribution $\mathcal{D}_t$, and the sequence $(\mathcal{D}_1, \dots, \mathcal{D}_T)$ may drift over time. Before observing the wake instances at round t , the learner uses past observations to construct a synthetic sleep training set $\tilde{\mathcal{T}}_t$, on which an inner AHSD algorithm $\mathcal{A}$ produces the deployed portfolio $P_t = \mathcal{A}(\tilde{\mathcal{T}}_t)$. Unlike offline AHSD, the sleep set is judged by the future wake performance of the portfolio it induces, rather than by its training performance alone.

The goal is to construct sleep environments that make the inner AHSD procedure produce portfolios with strong wake performance under distribution shift. Since a portfolio is evaluated by its best member on each instance, the natural continual objective is

$$\min_{\{\tilde{\mathcal{T}}_t\}_{t=1}^T} \sum_{t=1}^T \mathbb{E}_{x \sim \mathcal{D}_t} \left[\log \left(\min_{h \in P_t} f(x, h) + \epsilon \right) \right], \quad (2)$$

The logarithm makes the objective less sensitive to the absolute scale of instance difficulty, while preserving the ordering of positive objective values.

In implementation, SLEEP uses an equivalent reward written as improvement over a fixed reference heuristic h_0 :

$$r_t(P_t, x) = \log \frac{f(x, h_0) + \epsilon}{\min_{h \in P_t} f(x, h) + \epsilon}. \quad (3)$$

Because h_0 is fixed, maximizing expected reference improvement differs from minimizing the previous objective only by the additive term $\mathbb{E}_{x \sim \mathcal{D}_t} [\log(f(x, h_0) + \epsilon)]$, which is independent of the learned sleep set. This reward is therefore used for stable, interpretable feedback while leaving the underlying optimization problem unchanged.

3. Methodology

3.1. Sleep-Wake Framework

Overview. SLEEP alternates between a *sleep phase*, where it synthesizes training environments, and a *wake phase*, where the resulting portfolio is evaluated on the current real distribution. At round t , the learner has observed previous wake batches. It first constructs a sleep set $\tilde{\mathcal{T}}_t$, runs an inner AHSD algorithm $\mathcal{A}$, and deploys the portfolio $P_t = \mathcal{A}(\tilde{\mathcal{T}}_t)$. The wake batch $\mathcal{W}_t \sim \mathcal{D}_t$ is then used only for evaluation and outer-loop feedback.

Instance representation. The outer learner does not depend on a problem-specific solver state. Each instance x is summarized by a descriptor $\phi(x) \in \mathbb{R}^d$ that captures geometric and distributional properties of the instance. Let $(\psi_1, \dots, \psi_m)$ be the descriptor sequence in the recent wake window before round t . We use its empirical mean and temporal difference,

$$\bar{\phi}_t = \frac{1}{m} \sum_{i=1}^m \psi_i, \quad \Delta \phi_t = \frac{\psi_m - \psi_1}{\max(m-1, 1)}. \quad (4)$$

The policy state is then

$$s_t = [\bar{\phi}_t, \Delta \phi_t, t/T, \log(1 + n_t)], \quad (5)$$

where n_t is the number of wake observations seen so far. Empty histories use zero descriptor summaries. This state exposes both the current instance regime and the recent direction of distributional drift.

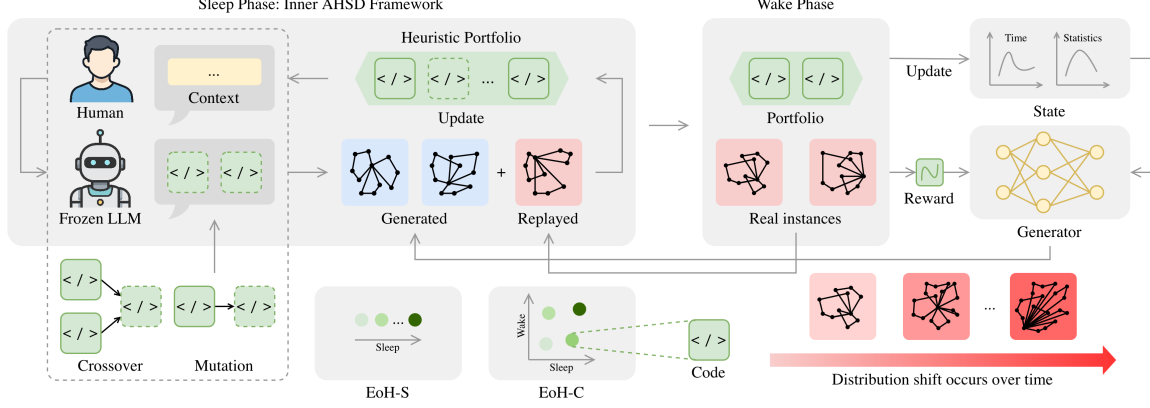


Figure 1. Overview of SLEEP for adapting heuristic portfolios under temporal distribution shift.

3.2. Learning Sleep Environments

Generator policy. The sleep environment is produced by a stochastic generator policy $\pi_\theta(z \mid s_t)$. An action z parameterizes a family of instance distributions, such as uniform, clustered, diagonal, or mixed point patterns in the current implementation. Let q_z denote the synthetic instance distribution induced by z . At round t , SLEEP samples actions $z_{t,1}, \dots, z_{t,B} \sim \pi_\theta(\cdot \mid s_t)$ and draws candidate instances $\hat{x}_{t,b} \sim q_{z_{t,b}}$.

Sleep-set construction. SLEEP allocates the finite sleep budget toward generated instances that are both diverse with respect to the sampled group and potentially hard for the current portfolio. At round t , for each sampled action $z_{t,b}$, we draw probe instances and summarize them by an average descriptor $\hat{\phi}_{t,b}$. The diversity contribution of action b is its average distance to the other sampled actions:

$$D_{t,b} = \frac{1}{B-1} \sum_{b' \neq b} \|\hat{\phi}_{t,b} - \hat{\phi}_{t,b'}\|_2. \quad (6)$$

When a previous portfolio P_{t-1} is available, hardness is estimated by disagreement among its members. For a representative generated instance $\hat{x}_{t,b}$, define $a_{t,b,h} = f(\hat{x}_{t,b}, h)$ for $h \in P_{t-1}$. We denote by $\mu_{t,b}$ and $\sigma_{t,b}$ the mean and standard deviation, respectively, of the scores $\{a_{t,b,h} : h \in P_{t-1}\}$ across portfolio members. The hardness score is

$$H_{t,b} = \frac{\sigma_{t,b}}{\mu_{t,b} + \epsilon}. \quad (7)$$

Generated training instances are allocated across actions with probability $p_{t,b} \propto \exp(\lambda_d D_{t,b} + \lambda_h H_{t,b})$. SLEEP then forms the sleep set $\tilde{\mathcal{T}}_t$ by combining generated instances sampled according to these weights with replayed wake instances from previous rounds.

Outer-loop update. After AHSD returns P_t , SLEEP evaluates it on the wake batch $\mathcal{W}_t$. The reward is the average

log improvement over the fixed reference heuristic h_0 :

$$R_t = \frac{1}{|\mathcal{W}_t|} \sum_{x \in \mathcal{W}_t} \log \frac{f(x, h_0) + \epsilon}{\min_{h \in P_t} f(x, h) + \epsilon}. \quad (8)$$

Since h_0 is fixed, this reward is equivalent to minimizing the log best-of-portfolio objective from the problem formulation. The generator is updated by a group-level policy-gradient step:

$$\nabla_\theta J \approx (R_t - b_t) \nabla_\theta \sum_{b=1}^B \log \pi_\theta(z_{t,b} \mid s_t) + \beta \nabla_\theta \mathcal{H}(\pi_\theta), \quad (9)$$

where b_t is an exponential moving-average baseline and $\mathcal{H}$ is an entropy bonus. The same delayed reward is assigned to all sampled actions because the entire group jointly determines the sleep set used to train P_t .

3.3. Inner Heuristic Portfolio Evolution

EoH-S backbone. The inner learner is based on EoH-S (Liu et al., 2026), an AHSD method that evolves a set of executable heuristics rather than a single best heuristic. Each candidate contains a short natural-language rationale and code. EoH-S initializes candidates from the reference heuristic and LLM-generated heuristics, then expands the population using crossover and mutation prompts. Each heuristic is evaluated on $\tilde{\mathcal{T}}_t$ to obtain its score vector $(f(x_1, h), \dots, f(x_N, h))$.

Portfolio-aware selection. For a training set $\mathcal{T}$, SLEEP converts candidate scores into per-instance rank-normalized utilities $u_i(h) \in [0, 1]$. A portfolio is scored by coverage:

$$C_{\mathcal{T}}(P) = \frac{1}{|\mathcal{T}|} \sum_{x_i \in \mathcal{T}} \max_{h \in P} u_i(h). \quad (10)$$

Greedy survival then adds heuristics that improve this coverage objective. This favors complementary heuristics that solve different subsets of the sleep set.

Table 1. Results with Ant Colony Optimization (ACO) as the backbone solver on size-100 routing problems, averaged over five runs. Imp. and Final report average improvement over vanilla ACO across the full stream and final window, respectively. Gap is the difference between improvement on generated sleep-training instances and held-out test instances; lower is better.

Method	Variant	TSP			CVRP			OVRP			LVRP			Avg. Rank
		Imp. ↑	Final ↑	Gap ↓	Imp. ↑	Final ↑	Gap ↓	Imp. ↑	Final ↑	Gap ↓	Imp. ↑	Final ↑	Gap ↓	
Baseline	Static EoH	7.19	8.73	2.30	7.19	10.98	2.72	10.46	12.18	2.88	7.61	8.79	2.64	6.08
SLEEP	Random generator	7.88	9.31	2.07	8.93	12.85	2.30	11.39	14.75	3.57	7.66	11.89	2.07	4.92
	w/o hardness reward	9.89	12.52	2.89	9.05	12.35	2.19	15.19	16.47	3.15	9.01	10.74	2.91	4.67
	w/o diversity reward	10.10	11.97	2.40	13.38	15.17	4.31	16.26	18.60	3.51	8.44	12.89	1.57	3.92
	w/o wake replay	11.12	13.08	3.10	12.04	14.66	3.58	18.65	21.20	4.71	11.72	14.18	3.33	4.17
	w/o counterfactual selection	10.67	11.80	2.16	12.55	14.92	2.58	20.98	23.04	2.84	10.30	14.70	1.95	2.67
	Full version	11.77	14.59	1.53	15.61	17.55	2.61	19.92	23.62	2.56	13.58	14.73	2.49	1.58

EoH-C: counterfactual portfolio evolution. We further propose EoH-C, a counterfactual extension of EoH-S for the continual setting. EoH-C augments the sleep objective with recent wake replay $\mathcal{C}_t = \{(x_j, w_j)\}$, where w_j is a recency weight. Here w_j decays exponentially with age: $w_j \propto \exp(-\rho(t - \tau_j))$ for observation time $\tau_j < t$. Its selection score combines sleep coverage, wake-replay coverage, and a regret penalty against the reference heuristic:

$$S(P) = C_{\mathcal{T}_t}(P) + \alpha \sum_{(x_j, w_j) \in \mathcal{C}_t} w_j \max_{h \in P} u_j(h) - \gamma \sum_{(x_j, w_j) \in \mathcal{C}_t} w_j \left[\frac{\min_{h \in P} f(x_j, h)}{f(x_j, h_0) + \epsilon} - 1 \right]_+ \quad (11)$$

EoH-C therefore keeps the AHSD search mechanism of EoH-S, but changes portfolio survival to account for counterfactual performance on recent real instances. This makes the inner learner less likely to overfit the generated sleep set and better aligned with the continual objective optimized by the outer SLEEP policy.

4. Experiments

Settings. We evaluate SLEEP in two solver settings. The main routing benchmark uses Ant Colony Optimization (ACO) on size-100 TSP, CVRP, OVRP, and LVRP instances, with results averaged over five runs. We also include a smaller Guided Local Search (GLS) study on TSP-20 to visualize early adaptation. Full experimental details are given in Appendix A.

Table 2. Interaction between the sleep-instance generator and the inner portfolio selector, averaged over five runs on TSP, CVRP, OVRP, and LVRP. Imp. and Final are improvement percentages; Gap is the generalization gap. ΔGen and ΔC compare generators and selectors, respectively.

Generator	Selector	Imp. ↑	Final ↑	Gap ↓	ΔGen Final	ΔC Final
Random	EoH-S	8.37	11.79	3.27	—	—
Learned	EoH-S	13.31	16.31	2.81	+4.52	—
Random	EoH-C	8.90	12.33	2.83	—	+0.54
Learned	EoH-C	15.23	18.12	2.33	+5.78	+1.80
Interaction gain		+1.38	+1.26	-0.04	difference-in-differences	

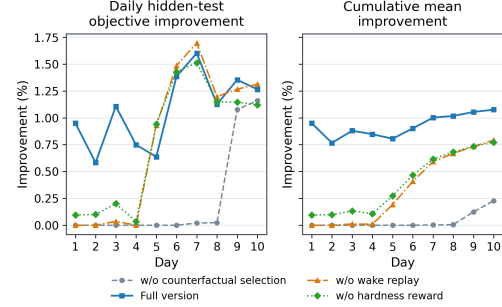


Figure 2. Applying SLEEP to Guided Local Search (GLS) on TSP-20 over the first 10 rounds. The left panel shows daily hidden-test improvement; the right shows cumulative mean improvement. The full version consistently outperforms ablations, showing the benefit of counterfactual selection, wake replay, and hardness reward.

Main routing results. Table 1 shows that the full SLEEP variant achieves the best average rank across ACO metrics and consistently improves final-window performance over Static EoH. This suggests that learning sleep environments is useful not only for TSP, but also for broader routing variants with different constraints. The ablations generally reduce improvement or increase the generalization gap, indicating that counterfactual selection, wake replay, diversity, and hardness provide complementary benefits.

Generator-selector interaction. Table 2 further shows that the learned generator and EoH-C reinforce each other. Learned sleep environments improve over random generation under both selectors, while EoH-C gives additional gains, especially when paired with the learned generator. This supports the view that SLEEP benefits from jointly improving what the inner learner trains on and how the resulting portfolio is selected.

Early adaptation. Figure 2 illustrates early adaptation on GLS for TSP-20. The full method maintains the strongest cumulative improvement, while ablations lag behind. This suggests that SLEEP can use limited wake feedback to adapt quickly, rather than waiting for a long stream of real instances before improving the portfolio.

References

- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pp. 1877–1901, 2020.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. d. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. doi: 10.48550/arXiv.2107.03374.
- Kiet, N. V. T., Dao, T., Tran, C. D., and Binh, H. T. T. Motif: Multi-strategy optimization via turn-based interactive framework. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pp. 37000–37008, 2026a.
- Kiet, N. V. T., Pham, B. D., Van Tung, D., Dao, T. C., and Binh, H. T. T. Back to the beginning of heuristic design: Bridging code and knowledge with llms. *arXiv preprint arXiv:2605.06123*, 2026b.
- Kojima, T., Gu, S. S., Reid, M., Matsuo, Y., and Iwasawa, Y. Large language models are zero-shot reasoners. *Advances in neural information processing systems*, 35: 22199–22213, 2022.
- Li, Y., Choi, D., Chung, J., Kushman, N., Schrittwieser, J., Leblond, R., Eccles, T., Keeling, J., Gimeno, F., Dal Lago, A., et al. Competition-level code generation with alphacode. *Science*, 378(6624):1092–1097, 2022. doi: 10.1126/science.abq1158.
- Liu, F., Xialiang, T., Yuan, M., Lin, X., Luo, F., Wang, Z., Lu, Z., and Zhang, Q. Evolution of heuristics: Towards efficient automatic algorithm design using large language model. In *International Conference on Machine Learning*, pp. 32201–32223. PMLR, 2024.
- Liu, F., Liu, Y., Zhang, Q., Xialiang, T., and Yuan, M. Eohs: Evolution of heuristic set using llms for automated heuristic design. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pp. 37090–37098, 2026.
- Parisi, G. I., Kemker, R., Part, J. L., Kanan, C., and Wermter, S. Continual lifelong learning with neural networks: A review. *Neural networks*, 113:54–71, 2019.
- Romera-Paredes, B., Barekatain, M., Novikov, A., Balog, M., Kumar, M. P., Dupont, E., Ruiz, F. J. R., Ellenberg, J. S., Wang, P., Fawzi, O., Kohli, P., and Fawzi, A. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024.
- Rozière, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X. E., Adi, Y., Liu, J., Sauvestre, R., Remez, T., et al. Code llama: Open foundation models for code, 2024.
- Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., and Yao, S. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- Wang, L., Zhang, X., Su, H., and Zhu, J. A comprehensive survey of continual learning: Theory, method and application. *IEEE transactions on pattern analysis and machine intelligence*, 46(8):5362–5383, 2024.
- Ye, H., Wang, J., Cao, Z., Berto, F., Hua, C., Kim, H., Park, J., and Song, G. Reevo: Large language models as hyper-heuristics with reflective evolution. *Advances in neural information processing systems*, 37:43571–43608, 2024.
- Zheng, Z., Xie, Z., Wang, Z., and Hooi, B. Monte carlo tree search for comprehensive exploration in llm-based automatic heuristic design. In *International Conference on Machine Learning*, pp. 78338–78373. PMLR, 2025.

Appendix: Supplementary Material

Learn in Sleep, Wake to Solve!

SLEEP: Simulated Future Learning Environments for Automated Evolution of Heuristic Portfolios

A. Experimental Settings

Benchmark problems and algorithms. We evaluate SLEEP on four routing problems: Traveling Salesman Problem (TSP), Capacitated Vehicle Routing Problem (CVRP), Open Vehicle Routing Problem (OVRP), and Duration-Limited Vehicle Routing Problem (LVRP). We follow the problem definitions and backbone solver interfaces from Kiet et al. (2026b) and ReEvo (Ye et al., 2024). In all experiments, the backbone solver is fixed; SLEEP only optimizes the heuristic component exposed to that solver. This makes the comparison focus on continual heuristic design rather than changes in the solver implementation.

Our main experiments apply SLEEP to Ant Colony Optimization (ACO). The learned heuristic is used as the edge desirability signal in ACO, while the vanilla ACO baseline uses the standard inverse-distance heuristic. Unless otherwise stated, ACO uses 30 ants and 100 construction iterations per instance. We additionally use Guided Local Search (GLS) as a secondary backbone for diagnostic visualization, where the learned function returns a penalty guide matrix.

Online evaluation protocol. Each run is an online stream of days. On day t , SLEEP first performs a sleep phase, where it generates synthetic training instances and optimizes a heuristic portfolio P_t . The resulting portfolio is then evaluated in a wake phase on real instances sampled from the current drifting distribution. Hidden test instances are sampled independently and are never used for replay, reward computation, or LLM prompting.

Table 3. Default online evaluation protocol.

Quantity	Default value
Number of days	30
Observed wake instances per day	5
Hidden test instances per day	20
Generated sleep instances per day	12
Replayed wake instances per day	6
Replay buffer size	200
Replay sampling	Recency-weighted, decay 0.1
Default drift schedule	Smooth uniform-to-clustered drift
Additional drift schedules	Abrupt and cyclic drift

Sleep-instance generator. The sleep generator is a small policy network that samples structured instance-generation actions. Each action specifies a geometric mode, a number of clusters, and continuous shape parameters. The sampled action is decoded into a two-dimensional point cloud; problem-specific fields such as demands or route constraints are then produced by the corresponding problem adapter.

Table 4. Action space and training settings for the sleep-instance generator.

Component	Setting
Geometry modes	Uniform, clustered, diagonal, mixture
Number of clusters	{2, 3, 4, 5, 6}
Continuous parameters	Spread, elongation, rotation, noise, mixture weight
Policy network	Two-layer MLP, hidden size 64
Group size per day	6 actions
Probe instances per action	2
Optimizer	Adam
Learning rate	5×10^{-3}
Entropy coefficient	0.02
Diversity weight	0.3
Hardness weight	0.5
Advantage baseline	Exponential moving average, coefficient 0.9
Advantage clipping	$[-5, 5]$

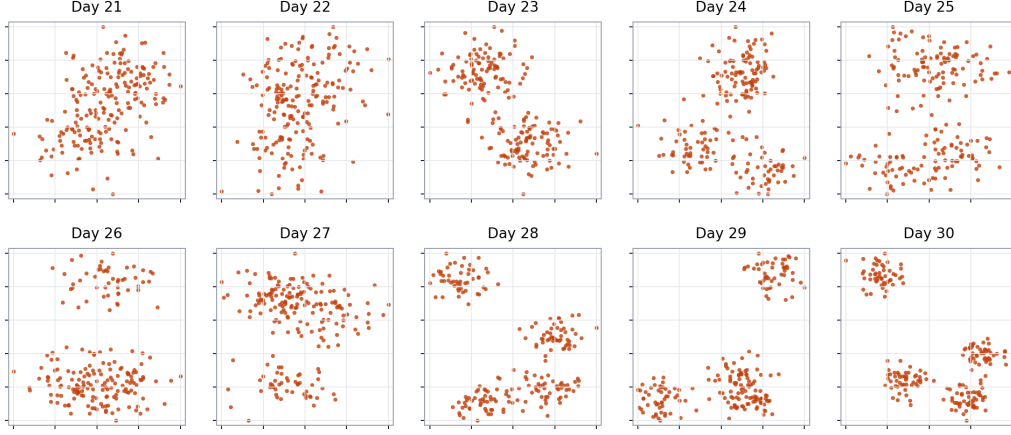


Figure 3. Distribution shift over the final 10 days under the default smooth-drift setting. Each panel shows one sampled TSP instance with 200 nodes; the spatial distribution gradually tightens into clustered structures from Day 21 to Day 30.

Drifting instance distributions. All online streams are generated from two-dimensional Euclidean point clouds. At day t , a real instance is sampled as $x \sim \mathcal{D}_t$, where $\mathcal{D}_t$ is induced by a geometric recipe over coordinates in the unit square. The same coordinate distribution is shared across routing problems; each problem adapter then constructs the problem-specific fields and solver input.

We use three drift schedules. Let $\alpha_t = t/(T - 1)$. The smooth schedule keeps a uniform regime in the first half, then gradually changes into a clustered regime:

$$\mathcal{D}_t = \begin{cases} \mathcal{Q}_{\text{unif}}, & \alpha_t < 1/2, \\ \mathcal{Q}_{\text{clus}}(r_t), & \alpha_t \geq 1/2, \end{cases} \quad r_t = 2\alpha_t - 1. \quad (12)$$

Here r_t controls the cluster geometry, with cluster spread decreasing and elongation increasing over time. The abrupt schedule uses three equal phases, $\mathcal{Q}_{\text{unif}} \rightarrow \mathcal{Q}_{\text{clus}} \rightarrow \mathcal{Q}_{\text{diag}}$. The cyclic schedule uses a mixture distribution with time-varying cluster weight

$$m_t = \frac{1 - \cos(2\pi(t \bmod p)/p)}{2}, \quad (13)$$

where $p = 20$ in our experiments.

B. Discussion on Diversity–Hardness Sleep Allocation

SLEEP allocates generated sleep instances according to two quantities: a diversity score $D_{t,b}$ and a hardness score $H_{t,b}$. This appendix provides a simple interpretation of this rule. The result should not be read as a guarantee that the resulting portfolio must improve on future wake distributions. Rather, it shows that the allocation rule is the optimizer of a natural entropy-regularized acquisition objective, and that the hardness term captures a portfolio-complementarity signal.

At round t , suppose the generator samples B candidate actions. Each action $z_{t,b}$ induces a synthetic instance distribution and is summarized by a descriptor vector $\hat{\phi}_{t,b}$. We define the diversity contribution of action b by

$$D_{t,b} = \frac{1}{B-1} \sum_{b' \neq b} \|\hat{\phi}_{t,b} - \hat{\phi}_{t,b'}\|_2. \quad (14)$$

When a previous portfolio P_{t-1} is available, we also estimate how much its members disagree on a representative generated instance $\hat{x}_{t,b}$. Let $a_{t,b,h} = f(\hat{x}_{t,b}, h)$ for $h \in P_{t-1}$, and let $\mu_{t,b}$ and $\sigma_{t,b}$ be the mean and standard deviation of these values across portfolio members. The hardness score is the coefficient of variation

$$H_{t,b} = \frac{\sigma_{t,b}}{\mu_{t,b} + \epsilon}. \quad (15)$$

The acquisition score used for allocation is $U_{t,b} = \lambda_d D_{t,b} + \lambda_h H_{t,b}$.

Entropy-regularized allocation. The softmax allocation used by SLEEP can be derived as the solution to an entropy-regularized acquisition problem. Consider choosing a sampling distribution p_t over the B candidate actions:

$$\max_{p_t \in \Delta_B} \sum_{b=1}^B p_{t,b} U_{t,b} + \tau \left(- \sum_{b=1}^B p_{t,b} \log p_{t,b} \right), \quad (16)$$

where Δ_B is the probability simplex and $\tau > 0$ controls the amount of exploration. The objective is strictly concave in p_t , so it has a unique maximizer.

The Lagrangian is

$$\mathcal{L}(p_t, \alpha) = \sum_{b=1}^B p_{t,b} U_{t,b} - \tau \sum_{b=1}^B p_{t,b} \log p_{t,b} + \alpha \left(\sum_{b=1}^B p_{t,b} - 1 \right). \quad (17)$$

Taking derivatives with respect to $p_{t,b}$ gives

$$U_{t,b} - \tau(\log p_{t,b} + 1) + \alpha = 0. \quad (18)$$

Therefore $p_{t,b} \propto \exp(U_{t,b}/\tau)$, and the unique solution is

$$p_{t,b} = \frac{\exp(U_{t,b}/\tau)}{\sum_{j=1}^B \exp(U_{t,j}/\tau)}. \quad (19)$$

The implementation corresponds to $\tau = 1$. Thus, the allocation rule can be viewed as maximizing expected diversity–hardness acquisition while retaining entropy, rather than greedily assigning all generated instances to the currently highest-scoring action.

Hardness as a complementarity signal. The hardness score is useful because AHSD is a portfolio-learning problem. It is not enough to find instances on which all heuristics perform uniformly poorly; a useful sleep instance should reveal behavioral differences among candidate heuristics, since such differences are what portfolio selection can exploit.

For a fixed generated instance $\hat{x}$ and portfolio P with M heuristics, write $a_h = f(\hat{x}, h)$, $\mu = M^{-1} \sum_{h \in P} a_h$, $\sigma^2 = M^{-1} \sum_{h \in P} (a_h - \mu)^2$, and $m = \min_{h \in P} a_h$. Assume $\mu > 0$. Since m is the best objective value attained by the current portfolio on $\hat{x}$, the gap $\mu - m$ measures how much better the best member is than the average member. The following inequality relates this gap to the hardness score $H = \sigma/\mu$:

$$\frac{\mu - m}{\mu} \geq \frac{H}{\sqrt{M-1}}. \quad (20)$$

To see this, let $\delta = \mu - m$. Every centered value satisfies $a_h - \mu \geq -\delta$, and the centered values sum to zero. Among all centered vectors with minimum $-\delta$ and zero sum, the largest possible squared norm is attained when $M-1$ entries equal $-\delta$ and the remaining entry equals $(M-1)\delta$. Hence

$$\sum_{h \in P} (a_h - \mu)^2 \leq M(M-1)\delta^2. \quad (21)$$

Dividing by M yields $\sigma^2 \leq (M-1)\delta^2$, so $\delta \geq \sigma/\sqrt{M-1}$. Dividing both sides by μ gives the claimed bound.

This bound shows that large hardness implies that the portfolio contains members with meaningfully different behavior on the generated instance: the best member must be separated from the portfolio average. Such instances are valuable for AHSD because the inner objective is best-of-portfolio coverage. They expose where some heuristics succeed and others fail, providing selection pressure for complementary heuristics rather than only for uniformly strong mean performance.

Implication for SLEEP. The two acquisition components play different roles. Diversity encourages generated sleep sets to cover distinct regions of the instance space, reducing the chance that the inner AHSD loop overfits to a narrow synthetic regime. Hardness emphasizes instances where the current portfolio has disagreement, which are precisely the instances where portfolio membership and complementarity matter. Since EoH-S (Liu et al., 2026) and our counterfactual variant

EoH-C select heuristics using instance-wise portfolio behavior, training on diverse and high-disagreement instances gives the inner learner more informative evidence for building complementary portfolios.

This argument is intentionally local. It justifies the sleep allocation rule as an acquisition strategy, not as a global performance guarantee. Future wake performance still depends on whether the generated sleep distribution is predictive of upcoming wake distributions and whether the inner AHSD procedure can synthesize heuristics that exploit the exposed structure.

C. Exact-Penalty View of EoH-C

We formalize the portfolio-selection step used by EoH-C. At round t , let $\mathcal{G}_t$ be the finite candidate population produced by the LLM evolutionary search, and let $\mathcal{P}_t = \{P \subseteq \mathcal{G}_t : |P| \leq M\}$ be the finite class of portfolios considered for survival. For any portfolio P , define its best-of-portfolio objective on instance x as $f_P(x) = \min_{h \in P} f(x, h)$. Let $u_x(h) \in [0, 1]$ be the rank-normalized utility of heuristic h on instance x , and define $u_x(P) = \max_{h \in P} u_x(h)$.

For the sleep set $\tilde{\mathcal{T}}_t$, define the sleep coverage

$$C_{\tilde{\mathcal{T}}_t}(P) = \frac{1}{|\tilde{\mathcal{T}}_t|} \sum_{x \in \tilde{\mathcal{T}}_t} u_x(P). \quad (22)$$

Let $\mathcal{C}_t = \{(x_j, w_j)\}_{j=1}^m$ be the counterfactual wake replay set, where $w_j \geq 0$ and $\sum_{j=1}^m w_j = 1$. Define replay coverage and replay regret by

$$C_{\mathcal{C}_t}(P) = \sum_{j=1}^m w_j u_{x_j}(P), \quad R_{\mathcal{C}_t}(P) = \sum_{j=1}^m w_j \left[\frac{f_P(x_j)}{f(x_j, h_0) + \epsilon} - 1 \right]_+. \quad (23)$$

EoH-C selects portfolios by maximizing the penalized score

$$S_t(P) = C_{\tilde{\mathcal{T}}_t}(P) + \alpha C_{\mathcal{C}_t}(P) - \gamma R_{\mathcal{C}_t}(P). \quad (24)$$

Interpretation. The regret term can be understood as a soft replay-safety mechanism rather than as a hard constraint. Since the coverage utilities are rank-normalized, both $C_{\tilde{\mathcal{T}}_t}(P)$ and $C_{\mathcal{C}_t}(P)$ lie in $[0, 1]$. Thus the positive contribution of coverage is bounded, while the replay-regret term can be made dominant by increasing γ . In this regime, portfolios that improve sleep coverage but substantially regress on recent wake instances become unattractive, even if they look strong on generated data. If a portfolio has zero replay regret, the penalty term vanishes and its score reduces to the combined sleep and replay coverage,

$$S_t(P) = C_{\tilde{\mathcal{T}}_t}(P) + \alpha C_{\mathcal{C}_t}(P). \quad (25)$$

EoH-C therefore behaves like standard portfolio coverage maximization on replay-safe candidates, while discouraging candidates that overfit the sleep set at the cost of recent real instances. This is the practical role of the counterfactual term: it biases survival toward heuristics that are useful on generated environments without forgetting what has recently worked during wake phases.

D. Examples of Generated Code

The heuristic portfolio found for TSP with GLS backbone:

```
# Heuristic No. 1
import random
import numpy as np
import pandas as pd
import scipy
import sklearn
import numpy as np
from sklearn.cluster import DBSCAN

def compute_penalty_guide(dist_mat):
    n_cities = dist_mat.shape[0]
    penalty_matrix = np.zeros_like(dist_mat)
```

```

# 1. Normalize distance matrix by average edge length
avg_dist = np.mean(dist_mat[dist_mat > 0])

# 2. Calculate distance-based penalty
for i in range(n_cities):
    for j in range(n_cities):
        if i != j:
            penalty_matrix[i, j] += (dist_mat[i, j] / avg_dist) # Higher penalty for
                                                                longer edges

# 3. Use DBSCAN to cluster the cities
coords = np.array(np.indices(dist_mat.shape)).reshape(2, -1).T # Create a coordinate
                                                                grid
clustering = DBSCAN(eps=1.5, min_samples=2).fit(coords) # Cluster based on proximity
labels = clustering.labels_

# 4. Penalize edges that cross clusters
for i in range(n_cities):
    for j in range(n_cities):
        if i != j:
            if labels[i] != labels[j]:
                penalty_matrix[i, j] += 1.5 # Arbitrarily choose a penalty for cross-
                                            cluster edges

# 5. Calculate local connectivity entropy
for i in range(n_cities):
    local_connections = [j for j in range(n_cities) if j != i and dist_mat[i, j] > 0]
    if local_connections:
        prob_dist = np.array([dist_mat[i, j] for j in local_connections]) / np.sum(
            dist_mat[i, local_connections])
        entropy = -np.sum(prob_dist * np.log(prob_dist + 1e-10)) # Adding a small
                                                                value to prevent log(0)

        for j in local_connections:
            penalty_matrix[i, j] += entropy

return penalty_matrix

```

```

# Heuristic No. 2
import random
import numpy as np
import pandas as pd
import scipy
import sklearn
import numpy as np
from sklearn.cluster import KMeans

def compute_penalty_guide(dist_mat: np.ndarray) -> np.ndarray:
    n = dist_mat.shape[0] # number of cities
    edge_priority = np.zeros_like(dist_mat)

    # Step 1: Perform clustering to identify local groups of cities
    kmeans = KMeans(n_clusters=min(n // 2, 8)) # Choose a reasonable number of clusters
    cluster_labels = kmeans.fit_predict(dist_mat)

    # Step 2: Calculate median distances within clusters
    median_distances = np.zeros(n)
    for cluster in range(max(cluster_labels) + 1):
        indices = np.where(cluster_labels == cluster)
        cluster_distances = dist_mat[indices][:, indices]
        median_distances[indices] = np.median(cluster_distances[cluster_distances > 0])

    # Step 3: Iterate over each edge to assess penalties

```

```

for i in range(n):
    neighbors_indices = np.where(cluster_labels == cluster_labels[i])[0] # Local
                                                                    neighbors
    local_distances = dist_mat[i, neighbors_indices]
    local_avg_distance = np.mean(local_distances)
    local_distance_variance = np.var(local_distances)

    for j in range(n):
        if i != j:
            edge_distance = dist_mat[i][j]

            # Penalize edges that connect different clusters
            if cluster_labels[i] != cluster_labels[j]:
                edge_priority[i][j] += edge_distance * 1.5 # Higher penalty for cross
                                                            -cluster edges

            # Penalize based on the deviation from local average distance
            if edge_distance > local_avg_distance + local_distance_variance:
                edge_priority[i][j] += edge_distance * 1.5 # Strong penalty for
                                                            outliers
            else:
                edge_priority[i][j] += edge_distance

# Step 4: Normalize edge_priority to a range of [0, 1]
if edge_priority.max() > 0:
    edge_priority = edge_priority / edge_priority.max()
return edge_priority

```

```

# Heuristic No. 3
import random
import numpy as np
import pandas as pd
import scipy
import sklearn
import numpy as np

def compute_penalty_guide(dist_mat: np.ndarray) -> np.ndarray:
    n_cities = dist_mat.shape[0]

    # Initialize the penalty guide matrix with zeros
    penalty_guide = np.zeros((n_cities, n_cities))

    # Pre-compute average distances, the median distance, and the maximum distance
    avg_distances = np.mean(dist_mat, axis=1)
    median_distances = np.median(dist_mat, axis=1)
    max_distance = np.max(dist_mat)

    # Calculate penalties focusing on long edges and their inconsistency with local
    # geometry
    for i in range(n_cities):
        for j in range(n_cities):
            if i != j:
                distance_ratio = dist_mat[i, j] / median_distances[i]
                penalty_weight = np.log1p(distance_ratio) * 2.0 # Adjusted scaling factor
                distance_diff = dist_mat[i, j] - avg_distances[i]

                # Emphasize longer edges and cross-cluster links
                if distance_diff > 0:
                    penalty_guide[i, j] = distance_diff * penalty_weight

    # Analyze neighborhood edges and adjust penalties dynamically
    for i in range(n_cities):
        neighbors = np.argsort(dist_mat[i])[:5] # Focus on the closest edges
        for j in range(n_cities):

```

```

    if i != j:
        if j not in neighbors:
            # Add harsher penalties for edges not in the local neighborhood
            penalty_guide[i, j] += 10.0 * (dist_mat[i, j] / max_distance) ** 1.3
            # Increased exponent for influence

        else:
            # Slightly reduce penalties for local neighbors, but not too aggressively
            penalty_guide[i, j] *= 0.75 # More gentle reduction factor

# Normalization step with a tighter thresholding based on average penalty
penalty_threshold = np.mean(penalty_guide[penalty_guide > 0]) * 0.03 # Lower
                                                                    # thresholding for stricter adjustment
penalty_guide[penalty_guide < penalty_threshold] = 0

# Normalize the penalty guide to be in the range [0, 1]
if np.max(penalty_guide) > 0:
    penalty_guide /= np.max(penalty_guide)

return penalty_guide

```

```

# Heuristic No. 4
import random
import numpy as np
import pandas as pd
import scipy
import sklearn
import numpy as np

def compute_penalty_guide(dist_mat: np.ndarray) -> np.ndarray:
    n_cities = dist_mat.shape[0]

    # Initialize the penalty guide matrix with zeros
    penalty_guide = np.zeros((n_cities, n_cities))

    # Compute local averages and standard deviations for enhanced penalty determination
    avg_distances = np.mean(dist_mat, axis=1)
    std_distances = np.std(dist_mat, axis=1)
    overall_avg_distance = np.mean(dist_mat[dist_mat > 0])
    max_distance = np.max(dist_mat)

    # Establish a base penalty influenced by the longer edges and variance
    for i in range(n_cities):
        for j in range(n_cities):
            if i != j:
                # Create relative penalty based on distance and variance
                distance_ratio = dist_mat[i, j] / overall_avg_distance if
                                                                    overall_avg_distance > 0 else
                                                                    0
                normalized_difference = (dist_mat[i, j] - avg_distances[i]) / np.maximum(
                                                                    std_distances[i], 1e-5)
                penalty_guide[i, j] = np.maximum(0, normalized_difference) * (
                                                                    distance_ratio ** 1.5) * 3.0

    # Implement additional penalties for edges connecting non-adjacent clusters presented
    # in distance matrix
    for i in range(n_cities):
        # Identify the closest neighborhoods for localized adjustments
        neighbors = np.argsort(dist_mat[i])[:5] # Focus on the top 5 nearest neighbors
        for j in range(n_cities):
            if i != j:
                if j not in neighbors:

```

```

        # Increase penalties for edges that connect outside the local cluster
        penalty_guide[i, j] += 10.0 * (dist_mat[i, j] / max_distance) ** 1.3
    else:
        # Decrease penalties for edges within the local cluster for leniency
        penalty_guide[i, j] *= 0.6 # More aggressive reduction for local
                                   connections

    # Normalize penalties and handle low penalty values with dynamic thresholding
    penalty_mean = np.mean(penalty_guide[penalty_guide > 0])
    threshold = penalty_mean * 0.1 # More adaptive thresholding for corrective measures
    penalty_guide[penalty_guide < threshold] = 0

    # Final normalization of penalty guide to ensure proper scaling
    max_penalty = np.max(penalty_guide)
    if max_penalty > 0:
        penalty_guide /= max_penalty

    return penalty_guide

```

```

# Heuristic No. 5
import random
import numpy as np
import pandas as pd
import scipy
import sklearn
import numpy as np
from sklearn.cluster import DBSCAN
from scipy.sparse.csgraph import shortest_path

def compute_penalty_guide(dist_mat: np.ndarray) -> np.ndarray:
    n = dist_mat.shape[0]
    penalty_mat = np.zeros_like(dist_mat)
    avg_distances = np.mean(dist_mat, axis=1)
    std_dev_global = np.std(dist_mat[dist_mat > 0])

    # DBSCAN clustering to find groups of cities
    clustering = DBSCAN(eps=1.0, min_samples=2).fit(dist_mat)
    labels = clustering.labels_

    for i in range(n):
        for j in range(n):
            if i != j:
                # Nearest neighbors excluding self-loop distance
                neighbors = np.delete(dist_mat[i], i)
                closest_neighbors = np.partition(neighbors, 5)[:5]
                avg_nearest_dist = np.mean(closest_neighbors)
                std_dev_neighbors = np.std(closest_neighbors)

                # Local deviation from average of nearest neighbors
                local_deviation = (dist_mat[i, j] - avg_nearest_dist) / (std_dev_neighbors
                                                                           + 1e-5)

                # Global adjustment based on average distances
                global_adjusted_deviation = (dist_mat[i, j] - avg_distances[j]) / (
                    std_dev_global + 1e-5)

                # Penalty for connecting isolated points or clusters
                isolation_penalty = 1.0 if labels[i] != labels[j] else 0.0

                # Final penalty calculation
                penalty_mat[i, j] = max(local_deviation + global_adjusted_deviation +
                                         isolation_penalty, 0) * np.
                                         log(dist_mat[i, j] + 1)

```

```
# Normalize penalties to range [0, 1]
max_penalty = np.max(penalty_mat)
min_penalty = np.min(penalty_mat[penalty_mat > 0]) if np.any(penalty_mat > 0) else 1.0

if max_penalty > min_penalty:
    penalty_mat = (penalty_mat - min_penalty) / (max_penalty - min_penalty + 1e-5)

return penalty_mat
```

E. Limitations, Scope, and Future Work

Scope of this workshop version. This workshop version introduces continual heuristic portfolio learning as a new problem setting for the automated heuristic design community. The goal is to present the sleep-wake formulation, show how synthetic environment generation can interact with portfolio evolution, and provide early evidence that the direction is promising. Our experiments are intentionally controlled: routing problems, fixed backbone solvers, and synthetic distribution shifts.

Limitations. The current study has several limitations:

- The drift processes are controlled and geometric, while real workloads may involve changes in constraints, graph topology, demand patterns, and solver budgets.
- The inner search relies on LLM-based evolution, so query cost and latency remain important practical concerns.
- The evaluation focuses mainly on best-of-portfolio solution quality; robustness, wall-clock cost, and portfolio-size sensitivity require further study.

Why this problem matters. Continual heuristic design connects AHD with the fact that optimization instances rarely arrive from one fixed distribution. Existing AHD methods show that LLMs can synthesize useful solver components, but most evaluations are offline. CHD asks a complementary question: how should a system use past execution experience to prepare for future instances that have not yet arrived? SLEEP is an initial answer based on using idle periods to construct training environments, rather than merely replaying past data.

Future work. The full version will develop the framework in several directions:

- evaluate on larger streams, heterogeneous graph sizes, stronger drift schedules, and real benchmark datasets;
- design richer generators for demands, time windows, edge weights, and other problem-specific structures;
- extend SLEEP beyond routing to scheduling, packing, graph partitioning, and other CO domains;
- add long-horizon memory for useful heuristics, failed designs, replay cases, and drift summaries.

Toward a fuller framework. This work should be read as a first step toward an agentic system for continual optimization. In a fuller framework, sleep would choose what to generate, replay, preserve, or modify, while wake would provide grounded execution feedback such as objective values, runtime, constraint violations, and failure cases. We believe this loop is a natural direction for optimization agents that improve through deployment.